Animaciones

Esta guía parte de la pantalla que dejó DataStore. El botón ya dice “Guardando…” mientras se escribe el archivo, y debajo aparece “Datos guardados en el teléfono” cuando `guardado` pasa a `true`. El contador, “Listo para enviar” y la lectura del nombre y del correo se quedan igual.

Acá ese cambio de estado se ve en movimiento:

- El botón cambia de color con `animateColorAsState`.
- El logo cambia de margen con `animateDpAsState`.
- El aviso entra y sale con `AnimatedVisibility`.
- Mientras se guarda, el botón muestra un loader.

La ruta `register_page`, el `ViewModel` y DataStore no se tocan.

1. El color del botón

Arriba de la `Column`, junto a `puedeEnviar`:

```
val colorBoton by animateColorAsState(
    targetValue = when {
        estado.guardando -> Color.Gray
        estado.guardado -> Color(0xFF2E7D32)
        puedeEnviar -> MaterialTheme.colorScheme.primary
        else -> Color(0xFF9E9E9E)
    },
    label = "colorBoton"
)
```

Imports:

```
import androidx.compose.animation.animateColorAsState
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.MaterialTheme
import androidx.compose.ui.graphics.Color
```

`animateColorAsState` recibe el color de destino y entrega el color de este fotograma. El `when` elige el destino según el estado:

- Gris mientras se guarda.
- Verde (`0xFF2E7D32`) cuando ya quedó escrito.
- `primary` del tema, el negro de `BlueDuoc`, cuando el formulario está listo.
- Un gris claro mientras faltan datos.

`label` es el nombre con el que Android Studio muestra esta animación en las herramientas.

En el botón, agrega `colors`. El texto de adentro sigue igual por ahora:

```
Button(
    onClick = {
        intentos++
        if (viewModel.validar()) {
            viewModel.guardar()
        } else {
            scope.launch {
                snackbarHostState.showSnackbar("Errores detectados. Revisa los campos.")
            }
        }
    },
    enabled = !estado.guardando,
    colors = ButtonDefaults.buttonColors(containerColor = colorBoton)
) {
    Text(if (estado.guardando) "Guardando..." else "Registrarse")
}
```

`ButtonDefaults.buttonColors` arma los colores del botón. `containerColor` es el fondo. Como `colorBoton` va cambiando en cada fotograma, el fondo se desplaza de un color al otro en vez de saltar.

Llena el formulario. El botón pasa del gris claro al color del tema. Al pulsar Registrarse, va al gris de “guardando” y después al verde.

2. El margen del logo

`animateDpAsState` funciona igual, con una distancia en `dp` en vez de un color.

Junto a `colorBoton`:

```
val paddingLogo by animateDpAsState(
    targetValue = if (estado.guardado) 8.dp else 40.dp,
    label = "paddingLogo"
)
```

Import:

```
import androidx.compose.animation.core.animateDpAsState
```

En la `Image` del logo, cambia el margen fijo por ese valor:

```
Image(
    painter = painterResource(R.drawable.logo),
    contentDescription = "Logo App",
    modifier = Modifier.padding(paddingLogo)
)
```

Al guardar, el aire alrededor del logo pasa de 40 dp a 8 dp. Al editar un campo, `guardado` vuelve a `false` y el margen regresa a 40 dp, también animado.

3. El aviso que aparece

Reemplaza el `if (estado.guardado)` de la guía de DataStore por `AnimatedVisibility`:

```
AnimatedVisibility(visible = estado.guardado) {
    Text("Datos guardados en el teléfono")
}
```

Import:

```
import androidx.compose.animation.AnimatedVisibility
```

`visible` decide si el contenido está en pantalla. Cuando pasa de `false` a `true`, el texto entra con una aparición corta. Cuando vuelve a `false`, sale. El `if` anterior lo ponía y lo quitaba de golpe.

4. El loader

Un loader dice que hay una tarea en curso. Aquí la tarea es la escritura en DataStore. El `delay(800)` del `ViewModel` deja ese círculo visible: sin esa espera, la escritura termina antes de que se note.

Imports:

```
import androidx.compose.foundation.layout.size
import androidx.compose.material3.CircularProgressIndicator
```

Dentro del botón, reemplaza el `Text` por esto:

```
if (estado.guardando) {
    CircularProgressIndicator(
        modifier = Modifier.size(20.dp),
        color = Color.White,
        strokeWidth = 2.dp
    )
} else {
    Text("Registrarse")
}
```

`CircularProgressIndicator` es el círculo que da vueltas. `size(20.dp)` lo deja chico, a la altura del texto del botón. `strokeWidth` es el grosor del trazo.

Mientras `guardando` es `true`, el botón muestra el círculo y queda deshabilitado. Al terminar, vuelve el texto “Registrarse”, el fondo pasa a verde y aparece el aviso.

Cómo queda RegisterScreen

Este es el archivo completo, con DataStore y las animaciones. Va en dos partes.

Parte 1, hasta el logo:

```
package cl.duoc.myapplication.ui

import androidx.compose.animation.AnimatedVisibility
import androidx.compose.animation.animateColorAsState
import androidx.compose.animation.core.animateDpAsState
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.size
import androidx.compose.material3.Button
import androidx.compose.material3.ButtonDefaults
import androidx.compose.material3.Checkbox
import androidx.compose.material3.CircularProgressIndicator
import androidx.compose.material3.HorizontalDivider
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.SnackbarHost
import androidx.compose.material3.SnackbarHostState
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.derivedStateOf
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.rememberCoroutineScope
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.runtime.setValue
import androidx.compose.ui.Alignment
import androidx.compose.ui.Modifier
import androidx.compose.ui.graphics.Color
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.lifecycle.viewmodel.compose.viewModel
import cl.duoc.myapplication.R
import cl.duoc.myapplication.navigation.Screen
import cl.duoc.myapplication.viewmodels.MainViewModel
import cl.duoc.myapplication.viewmodels.RegistroViewModel
import kotlinx.coroutines.launch

@Composable
fun RegisterScreen(mainViewModel: MainViewModel) {
    val viewModel: RegistroViewModel = viewModel()
    val estado by viewModel.estado.collectAsState()
    val snackbarHostState = remember { SnackbarHostState() }
    val scope = rememberCoroutineScope()
    var intentos by rememberSaveable { mutableStateOf(0) }
    val puedeEnviar by remember {
        derivedStateOf {
            estado.nombre.isNotBlank() &&
                estado.correo.contains("@") &&
                estado.aceptaTerminos
        }
    }
    val colorBoton by animateColorAsState(
        targetValue = when {
```

```
            estado.guardando -> Color.Gray
            estado.guardado -> Color(0xFF2E7D32)
            puedeEnviar -> MaterialTheme.colorScheme.primary
            else -> Color(0xFF9E9E9E)
        },
        label = "colorBoton"
    )
    val paddingLogo by animateDpAsState(
        targetValue = if (estado.guardado) 8.dp else 40.dp,
        label = "paddingLogo"
    )

    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )

        Column(
            modifier = Modifier
                .fillMaxSize()
                .padding(16.dp),
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center
        ) {
            Spacer(modifier = Modifier.weight(1F))

            Image(
                painter = painterResource(R.drawable.logo),
                contentDescription = "Logo App",
                modifier = Modifier.padding(paddingLogo)
            )
```

Parte 2, el resto de la columna:

```
        OutlinedTextField(
            value = estado.nombre,
            onValueChange = viewModel::onNombreChange,
            label = { Text("Nombre completo") },
            isError = estado.errores.nombre != null,
            supportingText = {
                estado.errores.nombre?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        OutlinedTextField(
            value = estado.correo,
            onValueChange = viewModel::onCorreoChange,
            label = { Text("Correo electrónico") },
            isError = estado.errores.correo != null,
            supportingText = {
                estado.errores.correo?.let { mensaje ->
                    Text(mensaje)
                }
            },
            modifier = Modifier.fillMaxWidth()
        )

        Row(verticalAlignment = Alignment.CenterVertically) {
            Checkbox(
                checked = estado.aceptaTerminos,
                onCheckedChange = viewModel::onAceptarTerminosChange
            )
            Text("Acepto los términos")
        }

        estado.errores.aceptaTerminos?.let { mensaje ->
            Text(mensaje)
        }

        HorizontalDivider(modifier = Modifier.padding(vertical = 16.dp))

        Text("Vista previa en tiempo real")
        Text("Nombre: ${estado.nombre}")
        Text("Correo: ${estado.correo}")
        Text("Términos aceptados: ${if (estado.aceptaTerminos) "Sí" else "No"}")

        Text("Intentos: $intentos")
        Text(if (puedeEnviar) "Listo para enviar" else "Faltan datos")

        Button(
            onClick = {
                intentos++
                if (viewModel.validar()) {
                    viewModel.guardar()
                } else {
                    scope.launch {
                        snackbarHostState.showSnackbar("Errores detectados. Revisa los
campos.")
                    }
                }
            },
            enabled = !estado.guardando,
            colors = ButtonDefaults.buttonColors(containerColor = colorBoton)
        ) {
```

```
                if (estado.guardando) {
                    CircularProgressIndicator(
                        modifier = Modifier.size(20.dp),
                        color = Color.White,
                        strokeWidth = 2.dp
                    )
                } else {
                    Text("Registrarse")
                }
            }

            AnimatedVisibility(visible = estado.guardado) {
                Text("Datos guardados en el teléfono")
            }

            Spacer(modifier = Modifier.weight(1F))

            Button(
                onClick = { mainViewModel.navigateTo(Screen.Home) }
            ) {
                Text("Volver")
            }

            SnackbarHost(hostState = snackbarHostState)
        }
    }
}
```

Qué probar

1. Con el formulario vacío, el botón se ve gris claro. Al completar nombre, correo con `@` y la casilla, el color pasa al del tema.
2. Pulsa **Registrarse**. Durante un momento el botón queda gris, muestra el círculo y no se puede pulsar.
3. Al terminar, el fondo pasa a verde, el logo se acerca y entra el texto “Datos guardados en el teléfono”.
4. Cambia una letra del nombre. El verde, el aviso y el margen del logo vuelven atrás.